

DEVOPS · KUBERNETES · MICROSERVICES

VProfile على Kubernetes

نشر تطبيق Java متعدد الطبقات كـ 4 microservices على كلاستر HA حقيقي —
بني الـ images بنفسنا، نأمنها، ونربطها ببعض خطوة بخطوة. دليل تعليمي كامل.

4

custom secure images

RHEL 9

masters & workers

6

nodes · 3 masters + 3 workers

ALnaqib

Docker Hub: [alnaqib](#) · GitHub: [yousefsalemW](#)
HA cluster fronted by HAProxy · k8s v1.36



الدليل مقسوم لـ microservice ورا الثاني، وكل جزء فيه: الـ Dockerfile بعد التأمين، أوامر البناء، الـ manifests، والأهم — الـ «ليه» ورا كل قرار.

Architecture & the VM→k8s map	00 نظرة عامة والمعمارية
Image security philosophy	01 فلسفة تأمين الـ Images
Prerequisites	02 التجهيزات
Persistent storage	03 التخزين: local-path-provisioner
MySQL / db01	04 Microservice · Database
cache / mc01	05 Microservice · Memcached
broker / rmq01	06 Microservice · RabbitMQ
Tomcat / app01	07 Microservice · Application
Build · Scan · Push	08 البناء والفحص والرفع
Deploy in order	09 النشر بالترتيب
Expose via HAProxy	10 الوصول عبر HAProxy
Verify & troubleshoot	11 التحقق واستكشاف الأخطاء
Wrap-up & next steps	12 الخلاصة والخطوات الجاية

نظرة عامة والمعمارية

00

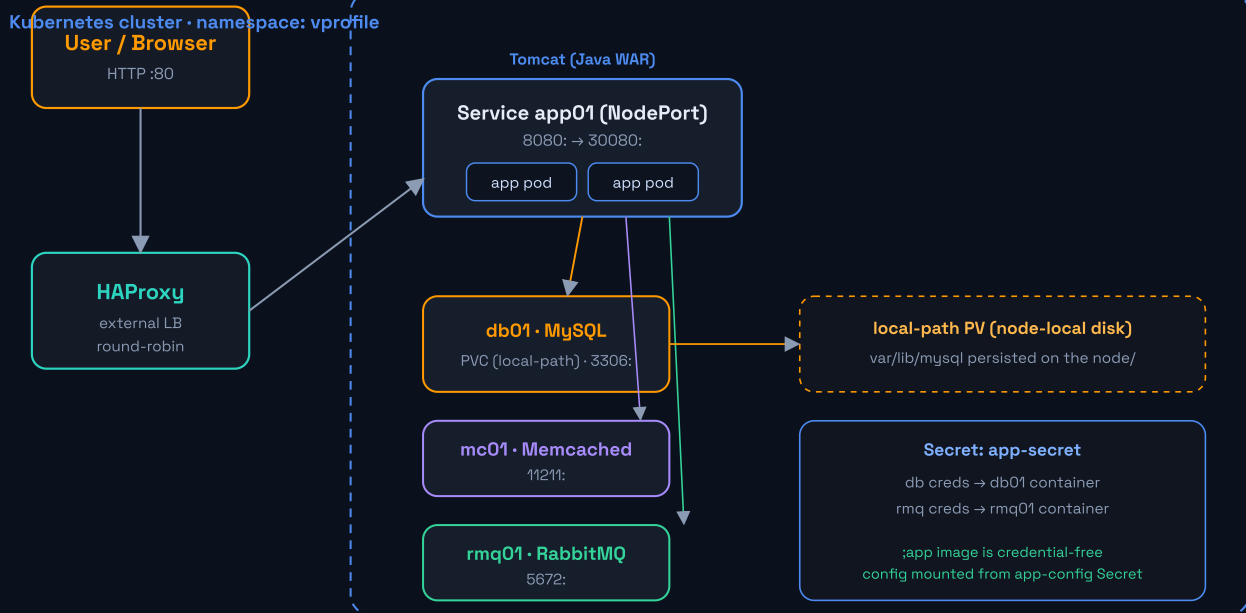
تطبيق Java Spring MVC يتكوّن من كذا طبقة يشتغل مع بعض: واجهة، تطبيق، قاعدة بيانات، كاش، وبروكر رسائل. في النسخة الأصلية كل **VProfile** طبقة كانت VM لوحدها بـ Vagrant. إحنا هنعوّلها لـ microservices على الكلاستر

الخريطة: من VM لـ Kubernetes

Port	Service	الشكل الجديد في k8s	Hostname	الطبقة القديمة (VM)
80	—	بيتشال — HAProxy بياخد دوره	web01	Nginx (web)
8080	app01	Deployment (2 replicas) + NodePort	app01	Tomcat (app)
3306	db01	Deployment + PVC + ClusterIP	db01	MySQL
11211	mc01	Deployment + ClusterIP	mc01	Memcached
5672	rmq01	Deployment + ClusterIP	rmq01	RabbitMQ

WHY • ليه كده؟

التطبيق أصلاً بيقرأ عناوين الخدمات من **application.properties** بأسماء زي **db01** و **mc01** و **rmq01**. وده بالظبط نظام الـ DNS في k8s: أي Service اسمه **db01** جوه namespace اسمه **vprofile** بيتحوّل لـ **db01.vprofile.svc.cluster.local**. يعني الـ service discovery شغال من غير ما نلمس الكود تقريباً — عشان كده أستخدم الـ Services لازم تبقى بالظبط **db01/mc01/rmq01**.



traffic: User → HAProxy → NodePort(app01) → app pods → db01 / mc01 / rmq01

NOTE • ملاحظة

في إعدادات Elasticsearch جوه الكود بس مفيش service ولا script ليها في المشروع الأصلي، فوهي **optional** وإحنا هتسيبها. التطبيق بيشتغل عادي من غيرها.

TIP • نصيحة

شلتنا (web01) لِن الـ **HAProxy** الموجود عندك يعمل نفس الدور بالظبط (reverse proxy + load balancing) بس على مستوى الكلاستر كله — فمفيش داعي لطبقة زيادة جوه k8s.

مين بيشتغل فين؟ • node placement

سؤال مهم: إتنا **مش** بنحدّد بإيدنا الـ pod يقعد على master ولا worker — ده شغل الـ scheduler. بس لازم تفهم النتيجة الافتراضية، ولازم تفرّق بين «الحاجة بتشتغل فين» و«الأمر بتنّفذه مين».

المكوّن	بيشتغل فين	مين بيقرّر
Control-plane (etcd · apiserver · scheduler · controller-mgr)	3 masters	kubeadm
kube-proxy + CNI (DaemonSet)	كل الـ 6 nodes	tolerates taint
pod الـ db01 + PVC	واحدة (متنبّئة) worker	scheduler + volume affinity
pods الـ mc01 / rmq01	على الـ workers	scheduler
pods الـ app (×2)	متوزّعة على الـ workers	scheduler
HAProxy	لوحدتها برّه الكلاستر VM	إنت
بناء ورفع الصور (build/push)	جهازك (workstation)	إنت
<code>kubectl apply</code> / <code>deploy.sh</code>	من master (فيها kubeconfig)	إنت

WHY • ليه كده؟

في كلاستر **kubeadm** الـ masters بييجي عليها taint اسمه `node-role.kubernetes.io/control-plane:NoSchedule`. ده معناه إن أي workload pod عادي **مش** هيتجول على master إلا لو عنده toleration. فطول ما إنت مشيلتش الـ taint — **كل pods الـ VProfile (db/mc/rmq/app) بتنزّل على الـ workers لوحدتها**، والـ masters بتفضل للـ control-plane بس.

NOTE • ملاحظة

استثناء: **kube-proxy** والـ CNI بيشتغلوا DaemonSet على **كل** الـ nodes (masters كمان) لإنهم بيتحقّلوا الـ taint. يعني الـ masters مش فاضية — بتشغل control-plane + الـ DaemonSets دي.

GOTCHA • خد بالك

الـ **db01** بالذات بيشتت على worker واحدة معيّنة تلقائيًا: عشان الـ PV local-path نوذي-لوكال، الـ provisioner بيحط **nodeAffinity** على الـ PV، فالـ pod بيرجع لنفس الـ node اللي الداتا عليها (تفاصيل أكثر في جزء 03).

تأكد من التوزيع الفعلي بإنك تشوف عمود الـ NODE:

BASH

verify placement ●●●

```
$ kubectl get pods -n vprofile -o wide # عمود NODE بيورّك كل NODE عمود
```

TIP • نصيحة

لو عايز تجرّ pod على node بعينها (نادراً ما تحتاج): استخدم **nodeSelector** أو **nodeAffinity** أو **tolerations** (عشان تشغل على master بشكل متعقّد). بس النصيحة: سيب الـ scheduler يشتغل، ماتحاربوش من غير سبب حقيقي.

قبل أي Dockerfile، علينا نتفق على المبادئ التي هنامشي عليها في كل صورة. دي مش تفاصيل تجميلية — دي التي بتفرق بين صورة صالحة للـ production وصورة خطر.

المبدأ	إحنا بنعمله إزاي
Multi-stage build	أدوات البناء (Maven/JDK) في stage لوحدها، والـ runtime فيه الـ <code>.war</code> بس — الصورة أصغر وسطح الهجوم أقل.
Non-root user	كل صورة بتشتغل بمستخدم عادي مش <code>root</code> (tomcat / mysql / memcache / rabbitmq).
Pinned versions	مقيش <code>:latest</code> — كل base image متثبت بـ tag محدد عشان النتيجة تبقى قابلة للتكرار.
No secrets in layers	صورة الـ app credential-free تمامًا — أي باسورد بيتشال وقت البناء، والإعدادات الحقيقية بتتحقن وقت التشغيل من Secret (mount).
Minimal base	للـ <code>alpine</code> للكاش والبروكر — أصغر وأنضف.
Scan (Trivy)	نقحص كل صورة للثغرات <code>HIGH/CRITICAL</code> قبل ما نرفعها.

WHY • ليه كده؟

«التأمين» مش خطوة واحدة في الآخر — ده قرار في كل سطر: base image إيه، مين اليوزر، فين الأسرار، إيه التي داخل الـ runtime. لما نطبق المبادئ دي من الأول، بنكون بنينا صورة نظيفة بدل ما نرفع واحدة وحشة.

NOTE • ملاحظة

الـ nodes كلها `RHEL 9 / amd64`، فبنبي بـ `platform linux/amd64` عشان نتجنب مشاكل الـ architecture لو بنبي من جهاز مختلف.

محتاجين نتأكد إن الكلاستر جاهز، وإن عندنا أدوات البناء على جهاز الـ workstation.

1) اتأكد إن الكلاستر صاحي

BASH

on master1

```
$ kubectl get nodes
NAME      STATUS   ROLES    VERSION
master1   Ready    control-plane  v1.36.2
master2   Ready    control-plane  v1.36.2
master3   Ready    control-plane  v1.36.2
worker1   Ready    worker        v1.36.2
worker2   Ready    worker        v1.36.2
worker3   Ready    worker        v1.36.2
```

2) أدوات البناء على الـ workstation

على جهاز البناء (اللاب بتاعك مثلاً) محتاج: `git`، `docker` (أو `podman` على RHEL)، و `trivy` للفحص.

BASH

workstation

```
# docker موجود؟
$ docker version

# trivy (للفحص) على RHEL/Fedora
$ sudo dnf install -y trivy # الرسمي binary أو من الـ
# (بتبني منه app + db صور) source نجيب الـ
$ git clone -b Master https://github.com/abdelrahmanonline4/sourcecodeseniorwr.git src
```

NOTE • ملاحظة

الـ nodes بتشغل الـ containers بـ `containerd` (مش `docker`)، وده مالوش دعوة ببناء الصور. إنت بتيني الصور على جهازك بـ `docker/podman` وترفعها Docker Hub، والـ nodes بتسحبها منه. الفصل ده مقصود.

3) الـ namespace

هنعزل كل المشروع في namespace اسمه `vprofile` (نفس الاسم اللي الكود متوقعه في الـ DNS).

YAML

k8s/00-namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: vprofile
  labels:
    app.kubernetes.io/part-of: vprofile
```

WHY • ليه كده؟

كل ال resources (Deployments/Services/Secrets/PVC) هتتحت جوّه ال namespace ده. ده بيدّينا عزل نظيف، وييحلي ال DNS الداخلي يشتغل بالشكل اللي التطبيق متوقعه: `db01.vprofile` بيتحوّل لـ `db01.vprofile.svc.cluster.local`

التخزين: local-path-provisioner

قاعدة البيانات محتاجة persistent storage — لو ال pod اتشال، الداتا لازم تفضل. الكلاستر المعمول يدوي بـ `kubeadm` غالبًا مفيوش StorageClass افتراضي، فالـ `PVC` هيفضل `Pending` للأبد. الحل: نرُكب `local-path-provisioner`.

التركيب (مرة واحدة على الكلاستر)

BASH

on master1

```
# من Rancher نرُكب الـ provisioner
$ kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/v0.0.31/deploy/local-path-storage.yaml

# بتاعه يشتغل pod نستنى الـ
$ kubectl -n local-path-storage rollout status deploy/local-path-provisioner

# الافتراضي StorageClass نخليه الـ
$ kubectl patch storageclass local-path \
  -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'

$ kubectl get sc
NAME                                PROVISIONER                DEFAULT
local-path (default)               rancher.io/local-path      true
```

WHY • ليه كده؟

الـ provisioner ده بيعمل PV تلقائي لكل PVC على قرص الـ node نفسها (تحت `/opt/local-path-provisioner`). بسيط وخفيف ومناسب جدًا لكلاستر تعليمي/داخلي من غير SAN ولا cloud storage.

GOTCHA • خد بالك

لإن التخزين `node-local`، الـ PV بيتربط بالـ node اللي اتعمل عليها، والـ provisioner بيحط `nodeAffinity` تلقائي عشان الـ pod يفضل بجدول على نفس الـ node. كفاية لـ DB بـ replica واحدة. لو عايز DB عالي الإتاحة بجد، ساعتها تحتاج حل تخزين شبكي (Longhorn / Ceph / NFS).

قاعدة البيانات 01 • db

الدور: قاعدة بيانات `accounts` التي فيها اليوزرز والصلاحيات، ومعمول لها seed من `db_backup.sql`. نستخدم MySQL 8.0 — التي التطبيق أصلاً متصقم عليه (ال connector في ال `pom` نسخته 8.0.22).

1) ال Dockerfile (بعد التأمين)

DOCKERFILE

images/db/Dockerfile

```
# =====
# VProfile DB (MySQL 8) - pre-seeded 'accounts' database
# build context = ../../src (the source-code repo root)
# =====
FROM mysql:8.0.40
LABEL maintainer="ALnaqib" \
      org.opencontainers.image.title="vprofile-db"

# database name is fixed; ALL passwords come at runtime from a k8s Secret
ENV MYSQL_DATABASE=accounts

# any *.sql here runs ONCE on first boot (empty data dir), in file order
# (db_backup.sql is a MySQL 5.7 dump; imports cleanly into MySQL 8.0)
COPY src/main/resources/db_backup.sql /docker-entrypoint-initdb.d/10-db_backup.sql

EXPOSE 3306
# NOTE: the official mysql image already runs as the unprivileged 'mysql' user
```

WHY • ليه كده؟

حاجتين مهمين: (1) ال `db_backup.sql` بنحطه في `/docker-entrypoint-initdb.d/` — أي ملف هناك بيتنفذ مرة واحدة أول ما ال `data dir` يبقى فاضي، فالجداول بتعمل تلقائي. (2) كل الباسوردات **مش** في الصورة — بيتجي وقت التشغيل من Secret عن طريق `MYSQL_*` environment variables.

2) ال Secret (المصدر الوحيد للباسوردات)

YAML

k8s/01-secret.yaml

```
# -----
# One Secret feeds the DB + RabbitMQ containers at runtime.
# IMPORTANT: db-user/db-password/rmq-user/rmq-password MUST match the
# --build-arg values you baked into the APP image, otherwise the app
# will authenticate with the wrong credentials.
# (stringData = plain text now, kubernetes base64-encodes it for you)
# -----
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
  namespace: vprofile
type: Opaque
stringData:
  db-root-password: "ChangeMe_Root_123"
  db-user: "admin"
  db-password: "admin123"
  rmq-user: "vprofile"
  rmq-password: "guest"
```

القيم دي لازم تطابق القيم اللي في الـ `app-config` Secret (اللي التطبيق بيقرأ منها إعداداته — هنشوفها في جزء 07). لو اختلفوا، الـ app هيفشل
يدخل بباسورد غلط ويفشل.

3) الـ PVC + Deployment + Service

YAML

k8s/02-db.yaml

```
# ===== PersistentVolumeClaim =====
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db-pvc
  namespace: vprofile
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: local-path      # provided by local-path-provisioner
  resources:
    requests:
      storage: 3Gi
---
# ===== Deployment =====
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vpro-db
  namespace: vprofile
spec:
  replicas: 1                      # a single DB pod owns the volume (RW0)
  selector:
    matchLabels: { app: vpro-db }
  strategy:
    type: Recreate                # never run 2 pods on one RW0 volume
  template:
    metadata:
      labels: { app: vpro-db }
    spec:
      containers:
        - name: vpro-db
          image: alnaqib/vprofile-db:1.0
          # force native password => old JDBC connects over plain TCP with no
          # caching_sha2 / public-key retrieval issues
          args: ["--default-authentication-plugin=mysql_native_password"]
          ports:
            - containerPort: 3306
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom: { secretKeyRef: { name: app-secret, key: db-root-password } }
            - name: MYSQL_USER
              valueFrom: { secretKeyRef: { name: app-secret, key: db-user } }
            - name: MYSQL_PASSWORD
              valueFrom: { secretKeyRef: { name: app-secret, key: db-password } }
          volumeMounts:
            - name: db-data
              mountPath: /var/lib/mysql
      volumes:
        - name: db-data
          persistentVolumeClaim:
            claimName: db-pvc
---
# ===== Service (name MUST be db01) =====
apiVersion: v1
kind: Service
metadata:
  name: db01                      # app talks to db01.vprofile:3306
  namespace: vprofile
spec:
```

```
selector: { app: vpro-db }
ports:
  - port: 3306
    targetPort: 3306
# ClusterIP (default) => internal only
```

WHY • ليه كده؟

strategy: Recreate و **replicas: 1** مقصودين: ال volume نوعه **ReadWriteOnce** يعني pod واحد بس يقدر يمسخه، لو استخدمننا ال **RollingUpdate** الافتراضي، k8s هيحاول يشغل pod جديد قبل ما يقفل القديم، والاتنين هيتخانقوا على نفس ال volume. **Recreate** بيقتل القديم الأول.

GOTCHA • خد بالك

لاحظ ال **args: ["--default-authentication-plugin=mysql_native_password"]** في ال MySQL8 Deployment. افتراضياً بيعمل اليوزر بـ **caching_sha2_password**، واللي على اتصال من غير TLS (زي جوه الكلاستر) بيرمي أحياناً **Public Key Retrieval is not allowed** مع ال JDBC. لما نجبر **mysql_native_password**، اليوزر **admin** بيتصل عادي على TCP من غير أي وجع. (ال option دي في MySQL 8.0؛ في 8.4 اتشالت وبقي بدلها **-- mysql-native-password=ON**).

NOTE • ملاحظة

ال **db_backup.sql** أصلاً dump من MySQL 5.7 — بيتقرأ في MySQL 8.0 عادي؛ أقصى حاجة تشوف تحذير charset بسيط (**utf8mb3** → **utf8**) وده مالوش تأثير.

الدور: Memcached ييخزن نتائج الاستعلامات المتكررة في الـ RAM عشان يخفف الحمل عن قاعدة البيانات. مفيش state دائم — لو راج، بيتبني ثاني من الـ .DB

1) Dockerfile ال

DOCKERFILE

images/memcached/Dockerfile

```
# =====
# VProfile CACHE (Memcached) - tiny hardened image
# build context = . (this folder)
# =====
FROM memcached:1.6.31-alpine
LABEL maintainer="ALnaqib" \
    org.opencontainers.image.title="vprofile-mc"

EXPOSE 11211
# -m 64 : cap RAM at 64 MB | -l 0.0.0.0 : listen on the pod network
# the base image already runs as the unprivileged 'memcache' user
CMD ["memcached", "-m", "64", "-l", "0.0.0.0", "-p", "11211"]
```

WHY • ليه كده؟

ضروري: افتراضياً memcached بيسمع على `127.0.0.1` بس، وده معناه إن أي pod تالي مش هيقدر يوصله. جوه k8s
الاتصال بيحي من عنوان الـ pod، فلانزم يسمع على كل الواجهات. و `-m 64` يحدد الـ RAM بـ 64MB عشان ماياكلش موارد الـ node.

2) Deployment + Service ال

YAML

k8s/03-memcached.yaml

```
# ===== Deployment =====
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vpro-mc
  namespace: vprofile
spec:
  replicas: 1
  selector:
    matchLabels: { app: vpro-mc }
  template:
    metadata:
      labels: { app: vpro-mc }
    spec:
      containers:
        - name: vpro-mc
          image: alnaqib/vprofile-mc:1.0
          ports:
            - containerPort: 11211
---
# ===== Service (name MUST be mc01) =====
apiVersion: v1
kind: Service
metadata:
  name: mc01
  namespace: vprofile
spec:
  selector: { app: vpro-mc }
  ports:
    - port: 11211
    targetPort: 11211
```


الدور: RabbitMQ يستقبل الرسائل من التطبيق ويسلمها (نمط producer/consumer). التطبيق يتصل عليه على 5672.

1) Dockerfile

DOCKERFILE

images/rabbitmq/Dockerfile

```
# =====
# VProfile BROKER (RabbitMQ) - management + hardened base
# build context = . (this folder)
# =====
FROM rabbitmq:3.13.7-management-alpine
LABEL maintainer="ALnaqib" \
    org.opencontainers.image.title="vprofile-rmq"

# 5672 = AMQP (used by the app) | 15672 = management UI
EXPOSE 5672 15672
# the login user/pass are injected at runtime via RABBITMQ_DEFAULT_USER/PASS
# (a real, non-'guest' user => works over the network with no loopback tricks)
```

WHY • ليه كده؟

في النسخة الأصلية اليوزر كان `guest`، بس RabbitMQ بيمنع `guest` من الاتصال من غير localhost لأسباب أمنية. بدل ما نفك القيد ده (حاجة وحشة أمنيًا)، بنعمل يوزر حقيقي عن طريق `RABBITMQ_DEFAULT_USER/PASS` من ال Secret — فيشغل عبر الشبكة عادي وبأمان.

2) Deployment + Service

YAML

k8s/04-rabbitmq.yaml

```
# ===== Deployment =====
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vpro-rmq
  namespace: vprofile
spec:
  replicas: 1
  selector:
    matchLabels: { app: vpro-rmq }
  template:
    metadata:
      labels: { app: vpro-rmq }
    spec:
      containers:
        - name: vpro-rmq
          image: alnaqib/vprofile-rmq:1.0
          ports:
            - containerPort: 5672 # AMQP
            - containerPort: 15672 # management UI
          env:
            - name: RABBITMQ_DEFAULT_USER
              valueFrom: { secretKeyRef: { name: app-secret, key: rmq-user } }
            - name: RABBITMQ_DEFAULT_PASS
              valueFrom: { secretKeyRef: { name: app-secret, key: rmq-password } }
---
# ===== Service (name MUST be rmq01) =====
apiVersion: v1
```

```
kind: Service
metadata:
  name: rmq01 # app talks to rmq01.vprofile:5672
  namespace: vprofile
spec:
  selector: { app: vpro-rmq }
  ports:
    - port: 5672
      targetPort: 5672
```

GOTCHA • خذ بالك

اليوزر/الباسورد بتوع rmq هنا لازم يطابقوا كمان القيم اللي في الـ `app-config` .Secret (rabbitmq.username / rabbitmq.password)

ده قلب المشروع: Tomcat 9 بيشتغل الـ `vprofile-v2.war`. الصورة دي أهم واحدة أمينيًا، وعشان كده عملناها **credential-free** بالكامل — مفيش أي باسورد جواها خالص.

1) الـ Dockerfile متعدد المراحل (multi-stage · secret-free)

DOCKERFILE

images/app/Dockerfile

```
# =====
# VProfile APP (Tomcat) - multi-stage, CREDENTIAL-FREE image
# build context = ../../src (the source-code repo root)
# NO passwords live in this image. The real application.properties
# is mounted at runtime from a k8s Secret (see k8s/05-app.yaml).
# =====

# ----- Stage 1: build + explode the WAR -----
FROM maven:3.9.9-eclipse-temurin-11 AS build
WORKDIR /workspace
COPY pom.xml .
COPY src ./src

# strip any credential from the properties that get compiled into the WAR,
# so nothing sensitive ever lands in an image layer
RUN set -eux; p=src/main/resources/application.properties; \
    sed -i "s#^jdbc.password=.*#jdbc.password=__SET_AT_RUNTIME__#" "$p"; \
    sed -i "s#^rabbitmq.password=.*#rabbitmq.password=__SET_AT_RUNTIME__#" "$p"; \
    mvn -q -B clean package -DskipTests; \
    mkdir -p /workspace/target/exploded && cd /workspace/target/exploded && \
    jar -xf ../vprofile-v2.war

# ----- Stage 2: minimal runtime -----
FROM tomcat:9.0.98-jre11-temurin-jammy
LABEL maintainer="ALnaqib" \
    org.opencontainers.image.title="vprofile-app" \
    org.opencontainers.image.source="https://github.com/yousefsaleMW"

RUN rm -rf /usr/local/tomcat/webapps/*
# ship the webapp EXPLODED so a Secret can be mounted over its properties file
COPY --from=build /workspace/target/exploded/ /usr/local/tomcat/webapps/ROOT/

# run as an unprivileged user (hardening)
RUN groupadd -r tomcat && useradd -r -g tomcat -d /usr/local/tomcat tomcat \
    && chown -R tomcat:tomcat /usr/local/tomcat
USER tomcat

EXPOSE 8080
CMD ["catalina.sh", "run"]
```

WHY · ليه كده؟

المرحلة الأولى (Maven) بتعمل حاجتين: (1) بتمسح أي باسورد من الـ `application.properties` قبل ما تبني (تحط مكانه `__SET_AT_RUNTIME__`) — فمفيش سر بيخس في أي layer من الصورة. (2) بعد ما تبني الـ `.war`، بتفكّه (explode) لفولدر — عشان نقدر نـ mount ملف الإعدادات الحقيقي فوقه وقت التشغيل. المرحلة الثانية (Tomcat) بتاخذ الفولدر المفكوك بس من غير Maven ولا JDK — صورة صغيرة، آمنة، ونضيفه من الأسرار.

2) الإعدادات الحقيقية: mount من Secret وقت التشغيل

الملف اللي جوه الصورة فاضي من الباسوردات. القيم الحقيقية (hosts + credentials) بتعيش في Secret اسمه `app-config` ، ونعمله mount بالظبط مكان `application.properties` جوه ال webapp — فالتطبيق يقرأه طبيعي وكأنه جزء منه.

YAML

k8s/05-app.yaml

```
# =====
# app-config: the FULL application.properties, kept OUT of the
# image and injected at runtime. This is the app's single
# source of truth for hosts + credentials.
# (put strong, real passwords here; they must match 01-secret.yaml)
# =====
apiVersion: v1
kind: Secret
metadata:
  name: app-config
  namespace: vprofile
type: Opaque
stringData:
  application.properties: |
    #JDBC Configuration
    jdbc.driverClassName=com.mysql.jdbc.Driver
    jdbc.url=jdbc:mysql://db01.vprofile:3306/accounts?useUnicode=true&characterEncoding=UTF-
8&zeroDateTimeBehavior=convertToNull
    jdbc.username=admin
    jdbc.password=admin123
    #Memcached
    memcached.active.host=mc01.vprofile
    memcached.active.port=11211
    memcached.standBy.host=mc01.vprofile
    memcached.standBy.port=11211
    #RabbitMQ
    rabbitmq.address=rmq01.vprofile
    rabbitmq.port=5672
    rabbitmq.username=vprofile
    rabbitmq.password=guest
    #Elasticsearch (optional - not deployed)
    elasticsearch.host=vprosearch01
    elasticsearch.port=9300
    elasticsearch.cluster=vprofile
    elasticsearch.node=vprofilenode
---
# ===== Deployment =====
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vpro-app
  namespace: vprofile
spec:
  replicas: 2                                # the app tier is stateless => scale freely
  selector:
    matchLabels: { app: vpro-app }
  template:
    metadata:
      labels: { app: vpro-app }
    spec:
      # ---- wait for the backing services before Tomcat starts ----
      initContainers:
        - name: wait-db
          image: busybox:1.36
          command: ['sh','-c','until nc -z db01 3306; do echo "waiting for db01"; sleep 3; done']
        - name: wait-mc
          image: busybox:1.36
          command: ['sh','-c','until nc -z mc01 11211; do echo "waiting for mc01"; sleep 3; done']
        - name: wait-rmq
          image: busybox:1.36
          command: ['sh','-c','until nc -z rmq01 5672; do echo "waiting for rmq01"; sleep 3; done']
      containers:
        - name: vpro-app
```

```

image: alnaqib/vprofile-app:1.0
ports:
  - containerPort: 8080
# inject the real config over the (credential-free) file baked in the image
volumeMounts:
  - name: app-config
    mountPath: /usr/local/tomcat/webapps/ROOT/WEB-INF/classes/application.properties
    subPath: application.properties
    readOnly: true
readinessProbe:
  httpGet: { path: /, port: 8080 }
  initialDelaySeconds: 40
  periodSeconds: 10
livenessProbe:
  tcpSocket: { port: 8080 }
  initialDelaySeconds: 60
  periodSeconds: 15
volumes:
  - name: app-config
    secret:
      secretName: app-config
---
# ===== Service (NodePort → reached by HAProxy) =====
apiVersion: v1
kind: Service
metadata:
  name: app01
  namespace: vprofile
spec:
  type: NodePort
  selector: { app: vpro-app }
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 30080 # every node exposes :30080 → app pods

```

• WHY • ليه كده؟

الـ Spring في VProfile (نسخة 4.2) بيحفل `classpath:application.properties` ، الي بيتترجم لـ `WEB-INF/classes/application.properties` جوة الـ `webapp`. عشان كده فُكِّنا الـ `WAR` في الصورة، فالمسار ده بقى موجود كملف فعلي — والـ `subPath` mount بيسبدل الملف الفاضي بالملف الحقيقي من الـ `Secret`. النتيجة: **نفس الصورة بالضبط** تشتغل في `dev` و `prod`، والفرق الوحيد هو الـ `Secret` المحقون، ده معنى «الصورة generic ومحدث يعرف منها سر».

• TIP • نصيحة

الفايدة الأمنية الحقيقية: قبل كده الباسورد كان بيتخز جوه الـ `war` — أي حد يسحب الصورة يطلع، دلوقتي الصورة نظيفة تمامًا، والباسورد بقى في الـ `app-config` `Secret` بس — وهنا الـ `RBAC` والـ `encryption-at-rest` بقى ليهم معنى فعلي.

• NOTE • ملاحظة

القيم في `app-config` (`rabbitmq.password` / `jdbc.password`) لازم تطابق قيم الـ `app-secret` الي بتغذي حاويات الـ `db` والـ `rmq`، فيه لسه نسختين للباسورد بس **اللاتنين جوه الكلاستر ومحميين** (مش في صورة). الخطوة الي بعدها لو حبيت تشيل الازدواجية: تخلي الـ `properties` تقرا من الـ `app-secret` مباشرة. `{{ENV}}`

3) الترتيب والـ probes

• WHY • ليه كده؟

الـ `initContainers` بتستنى `db01` و `mc01` و `rmq01` بيقوا شغالين (لـ `nc -z`) قبل ما Tomcat يقوم، من غيرها، التطبيق ممكن يقوم ويحاول يتصل بقاعدة بيانات لسه مش جاهزة ويعمل `CrashLoopBackOff`. وده معنى «الترتيب» في الـ `microservices` — التبعية بتحل بذكاء مش بالصدفة.

• TIP • نصيحة

الـ `readinessProbe` يتمنع HAProxy إنه ييغت طلبات لـ pod لسه بيسخن (Tomcat بياخد وقت في الإقلاع). و `replicas: 2` عشان الطبقة دي stateless فنقدر نوّزع الحمل.

VProfile on Kubernetes · alnaqib/*

ALnaqib

دلوقة نبي الأربعة صور، نفحصهم بـ Trivy، ونرفعهم على `alnaqib/*`

BASH

build-and-push.sh

```
#!/usr/bin/env bash
# Build, scan and push the 4 VProfile images to Docker Hub (alnaqib/*)
# Run from a workstation that has: git, docker (or podman), trivy
set -euo pipefail

DHUB="alnaqib"          # your Docker Hub username
TAG="1.0"
PLATFORM="linux/amd64" # RHEL 9 nodes are amd64

# 0) get the source repo (app + db images build FROM it)
[ -d src ] || git clone -b Master https://github.com/abdelrahmanonline4/sourcecodeseniorwr.git src

# 1) build
docker build --platform "$PLATFORM" -t "$DHUB/vprofile-app:$TAG" -f images/app/Dockerfile      src/
docker build --platform "$PLATFORM" -t "$DHUB/vprofile-db:$TAG" -f images/db/Dockerfile       src/
docker build --platform "$PLATFORM" -t "$DHUB/vprofile-mc:$TAG" -f images/memcached/Dockerfile images/memcached/
docker build --platform "$PLATFORM" -t "$DHUB/vprofile-rmq:$TAG" -f images/rabbitmq/Dockerfile images/rabbitmq/

# 2) scan (fail the build on HIGH/CRITICAL if you drop '|| true')
for s in app db mc rmq; do
  echo "= scanning $s ="
  trivy image --severity HIGH,CRITICAL --ignore-unfixed "$DHUB/vprofile-$s:$TAG" || true
done

# 3) push
docker login -u "$DHUB"
for s in app db mc rmq; do docker push "$DHUB/vprofile-$s:$TAG"; done
echo "done."
```

WHY · ليه كده؟

صورة الـ app دلوقة مفيهاش أي باسورد — فمفيش `-- build-arg` لأسرار خالص، ومش محتاج تبنيها تاني لو غيّرت باسورد. التغيير يحصل في الـ `app-config` Secret بس وتعمل `rollout restart` للـ deployment. لاحظ برضه `platform linux/amd64` عشان الـ nodes كلها RHEL 9 / amd64.

NOTE · ملاحظة

لو خليت الـ repos **private** على Docker Hub، الـ nodes مش هتقدر تسحب الصور غير بـ `imagePullSecret`. اعمله كده:

```
kubectl -n vprofile create secret docker-registry regcred --docker-username=alnaqib --docker-password=...
```

وبعدين ضيف `imagePullSecrets: [name: regcred]` في الـ pod spec.

TIP · نصيحة

الـ `trivy` بيوقف عند `HIGH/CRITICAL`. لو عايزه يفشل الـ build فعلاً (بدل ما يكفل)، شيل `|| true` من السطر بتاعه.

الترتيب مهم: namespace ← secret ← الخدمات الخلفية (db, mc, rmq) ← التطبيق. الـ initContainers لو حصل تداخل بسيط، بس الأنضف إننا نمشي بالترتيب.

BASH

deploy.sh

```
#!/usr/bin/env bash
# Deploy VProfile to the cluster (run where kubectl is configured, e.g. master1)
set -euo pipefail

# 1) storage class (once per cluster)
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/v0.0.31/deploy/local-path-storage.yaml
kubectl -n local-path-storage rollout status deploy/local-path-provisioner
kubectl patch storageclass local-path \
  -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'

# 2) app stack (order matters: ns → secret → backing services → app)
kubectl apply -f k8s/00-namespace.yaml
kubectl apply -f k8s/01-secret.yaml
kubectl apply -f k8s/02-db.yaml
kubectl apply -f k8s/03-memcached.yaml
kubectl apply -f k8s/04-rabbitmq.yaml
kubectl apply -f k8s/05-app.yaml

# 3) watch it come up
kubectl -n vprofile get pods -w
```

تتبع الإقلاع

BASH

on master1

```
$ kubectl -n vprofile get pods
NAME          READY   STATUS    RESTARTS
vpro-db-xxxx  1/1     Running   0
vpro-mc-xxxx  1/1     Running   0
vpro-rmq-xxxx 1/1     Running   0
vpro-app-xxxx 0/1     Init:2/3   0      # لمسه بيستنى
vpro-app-xxxx 1/1     Running   0      # جاهز
```

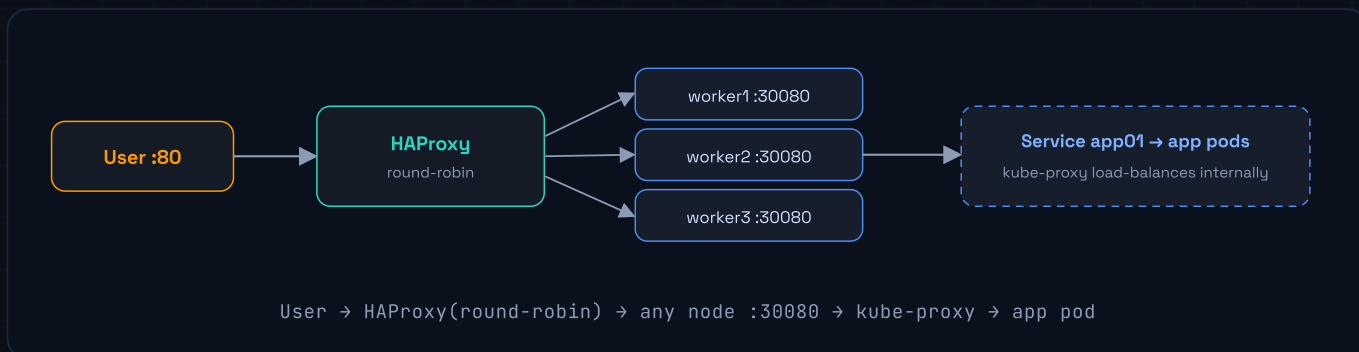
NOTE · ملاحظة

أول مرة، صورة الـ db بتعمل seed للجداول من `db_backup.sql` — ممكن تاخذ شوية ثواني. الـ app هيفضل في `Init` لحد ما db/mc/rmq يردّوا.

HAProxy عبر Service

10

الـ `app01` Service نوعه NodePort — يعني كل node في الكلاستر بتفتح البورت `30080` ويتوصله لـ pods التطبيق، الـ HAProxy بيوتج الطلبات على الـ `nodes`.



الإعدادات الحالي عندك (k8s-api)

الـ HAProxy عندك أصلاً بيعمل load-balancing للـ Kubernetes API على الـ 3 masters. إحنا هنبي على ده — نضيف frontend/backend جديد للـ app، من غير ما نلمس اللي شغال.

HAPROXY

etc/haproxy/haproxy.cfg (current)/

```

#-----
# GLOBAL
#-----
global
    log /dev/log local0
    log /dev/log local1 notice
    maxconn 4000
    user haproxy
    group haproxy
    daemon

#-----
# DEFAULTS (note: mode tcp is the default here)
#-----
defaults
    log global
    mode tcp
    option tcplog
    option dontlognull
    timeout connect 10s
    timeout client 86400s
    timeout server 86400s
    timeout tunnel 86400s
    retries 3

#-----
# FRONTEND - Kubernetes API
#-----
frontend k8s-api
    bind *:6443
    mode tcp
    default_backend k8s-masters

#-----
# BACKEND - Kubernetes Control Plane Nodes
#-----
backend k8s-masters
    mode tcp
    balance roundrobin
    option tcp-check
  
```

```
server master1 10.0.12.138:6443 check inter 2s fall 3 rise 2
server master2 10.0.12.150:6443 check inter 2s fall 3 rise 2
server master3 10.0.12.186:6443 check inter 2s fall 3 rise 2
```

GOTCHA • خذ بالك

لاحظ إن الـ `defaults` عندك `mode tcp` (مضبوط لـ `k8s-api` على 6443). عشان كده الـ frontend/backend الـ app لازم يحدّدوا `mode http` صراحة — وده معمول في الإعدادات تحت.

1) هات عناوين الـ nodes

BASH

on master1

```
$ kubectl get nodes -o wide
# worker1/2/3 لـ INTERNAL-IP خد عمود
```

2) ضيف الإعدادات في HAProxy

HAPROXY

etc/haproxy/haproxy.cfg (append)/

```
# -----
# Add to /etc/haproxy/haproxy.cfg (or /etc/haproxy/conf.d/)
# on the existing HAProxy VM, then: systemctl reload haproxy
# Replace the IPs with: kubectl get nodes -o wide (INTERNAL-IP)
# -----
frontend vprofile_http
    bind *:80
    mode http
    default_backend vprofile_app_nodes

backend vprofile_app_nodes
    mode http
    balance roundrobin
    option httpchk GET /login
    http-check expect status 200
    server worker1 10.0.x.x:30080 check
    server worker2 10.0.x.x:30080 check
    server worker3 10.0.x.x:30080 check
```

3) افحص وأعد التحميل

BASH

on HAProxy VM

```
# افحص الإعدادات الأول
$ sudo haproxy -c -f /etc/haproxy/haproxy.cfg
Configuration file is valid

# أعد التحميل من غير قطع
$ sudo systemctl reload haproxy
```

WHY • ليه كده؟

الـ `option httpchk GET /login` بيخّلي HAProxy يعمل health check على كل node. مهم إننا نفحص `/login` مش `/` : التطبيق بيرد على `/` لصفحة الـ login، فلو فحصنا `/` وطلبنا HAProxy، status 200 هيعتبر كل الـ nodes **DOWN** ويرجع **503**. صفحة `/login` بيرجع **200** مباشرة، فالفحص يعطي صح.

TIP • نصيحة

تأكد إن كل حاجة شغالة، ونجيز نفسنا لأشهر المشاكل.

BASH

on master1

```
$ kubectl -n vprofile get all
$ kubectl -n vprofile get pvc,secret

# لوغات أي خدمة
$ kubectl -n vprofile logs deploy/vpro-app
$ kubectl -n vprofile logs deploy/vpro-db

# لو فيه مشكلة pod تفاصيل
$ kubectl -n vprofile describe pod <pod-name>

# اختبار سريع من جوه الكلاستر
$ kubectl -n vprofile run t --rm -it --image=busybox:1.36 -- \
  sh -c "nc -z db01 3306 && echo db-ok"
```

جدول أعطال سريع

العرض	السبب الغالب	التشخيص / الحل
Pod في ImagePullBackOff	اسم/تاج الصورة غلط أو repo خاص	<code>kubectl describe pod ...</code> : اتأكد من <code>almaqib/vprofile-*</code> ومن <code>regcred</code>
PVC في Pending	مفيش StorageClass افتراضي	<code>kubectl get sc 03</code> : رجّع لجزء 03
vpro-db CrashLoop	قديمة فيها داتا بباسورد مختلف PVC	امسح الـ PVC وأعد النشر (بتفقد الداتا) أو صحّح الـ Secret
Init عالق في app	مش جاهزين db/mc/rmq	<code>kubectl -n vprofile get pods : logs</code> للخدمة الناقصة
يبرجع 503 HAProxy	nodes down أو غلط الـ NodePort	<code>curl http://NODE_IP:30080</code> من الـ HAProxy VM
الصفحة بتفتح بس login بيفشل	باسورد الـ app-config ≠ باسورد الـ DB Secret	طابق <code>jdbc.password</code> في <code>app-config</code> مع <code>db-password</code> في <code>app-secret</code>

TIP • نصيحة

بعد ما كله يبقى **Running**، افتح `http://HAPROXY_IP/` في المتصفح — المفروض تلاقي صفحة دخول VProfile. جرب تسجل يوزر جديد؛ لو اتسجل يبقى الـ DB شغال، ولو الصفحة اشتغلت من غير أخطاء يبقى الاتصال بالكاش والبروكر تمام.

كده حوّلنا **VProfile** من 5 VMs لـ **microservices 4** على كلاستر HA حقيقي: بنينا صور آمنة بنفسنا، ربطنا الأسرار صح، رتبنا التبعيات بالـ `initContainers`، حوّنا حالة الـ DB، وفتّحنا الباب للعالم عبر `HAProxy`.

اللي اتعلمناه (المفاهيم)

- الـ `Service name` = الـ `DNS name` — عشان كده أسماء الخدمات مش عشوائية.
- الـ `multi-stage build` ييفصل أدوات البناء عن الـ `runtime` = صورة أصغر وأمن.
- الصور **credential-free**: الأسرار بتتحقن وقت التشغيل من `Secret` (أو `mount`)، مش جوه طبقات الصورة.
- التبعية بين الخدمات بتتحل بـ `initContainers`، مش بالحظ.
- الحالة (state) محتاجة `PVC`: الكاش لئ.

خطوات للتطوير (production-grade)

- داخل الكلاستر HTTPS لو حبيت (`nginx-ingress` / `cert-manager`) بدل `Ingress + TLS`.
- **شيل ازدواجية الباسورد**: خّي الـ `app-config` يقرأ `{ENV}` من الـ `app-secret` (Spring 4.2 بيدعمها) بدل نسخة تانية من الباسورد — مصدر واحد للحقيقة.
- يمنع مين يقرأ الأسرار — دلوقتي بقي ليهم معنى لإن الصورة نضيفه `etcd + RBAC` على الـ **Encryption at rest**.
- حسب الحمل `app` توسيع تلقائي لطبقة الـ **HPA**.
- العادي `Secret` بدل الـ **Secrets manager** (`Vault` / `Sealed Secrets`).
- **Monitoring**: `Prometheus + Grafana` خدمات.

TIP · نصيحة

كل الملفات (`Dockerfiles + manifests + scripts`) موجودة في الـ `bundle` اللي مرفوع مع الـ PDF — تقدر تستخدمها مباشرة بعد ما تعدّل الـ IPs والباسوردات.

ALnaqib

DevOps • Kubernetes • Cairo • 2026